

WHAT I LEARNED FROM THIS ASSIGNMENT

1. INTRODUCTION

This assignment helped me understand how Flutter applications handle dynamic data received from an API. Instead of displaying only fixed or hard-coded information, the application fetches product data asynchronously and updates the user interface according to the result.

Through this assignment, I learned about creating a data model, using a Mock API, working with Future and async/await, managing different UI states, handling errors, using retry functionality, handling null values, and displaying dynamic data using ListView.builder.

The assignment also helped me understand that a real application needs to handle different situations instead of assuming that an API request will always be successful.

2. UNDERSTANDING DATA MODELS

I learned the importance of creating a model class to represent application data.

In this assignment, the Product class represents product information using three properties: id, name, and price.

The price field is defined as a nullable double. This means that the price can either contain a value or be null. This is useful because API data may sometimes have missing or unavailable values.

For example, the application contains a product whose price is null. Instead of causing an error, the application checks the value and displays "Price not available".

This helped me understand how models provide a structured way to work with data in Flutter.

3. LEARNING ASYNCHRONOUS PROGRAMMING

One of the most important concepts I learned was asynchronous programming.

API requests take time because data normally has to be retrieved from an external server. Flutter therefore needs to handle these operations asynchronously.

The Mock API uses Future.delayed() to simulate a two-second network delay. This helped me understand that an application cannot assume that data will be available immediately.

I learned how Future represents a value that will become available later and how asynchronous operations can be handled using async and await.

The basic flow I learned was:

Start request → Wait for response → Receive data → Update UI

The Mock API uses a Future-based fetchProducts() method to simulate this behavior.

4. LEARNING LOADING, ERROR AND SUCCESS STATES

I learned that an API-driven screen can have multiple states.

The application uses an `isLoading` variable to identify when data is being fetched. During this state, a `CircularProgressIndicator` is displayed.

If the API request fails, an error state is displayed with an error message and a Retry button.

If the request succeeds, the product data is displayed.

The application also handles a null or empty response by displaying "No products available".

Therefore, I learned to think about a screen in terms of different states:

1. Loading
2. Error
3. Null or Empty
4. Success

This is important because a professional application should provide an appropriate UI for each possible situation.

5. LEARNING ERROR HANDLING AND RETRY

I learned how try-catch can be used to handle errors during asynchronous operations.

The Mock API intentionally generates an exception during the first request. The application catches this exception instead of allowing the application to crash.

When an error occurs, `errorMessage` is updated and the loading state is stopped.

A Retry button is then displayed. When the user presses it, `fetchData()` is called again.

This taught me that error handling is not only about preventing crashes. It should also provide the user with a way to recover from temporary problems.

The Mock API was useful for learning this because it deliberately simulates a failed first request.

6. LEARNING STATE MANAGEMENT WITH `StatefulWidget`

I learned why `StatefulWidget` is useful when working with dynamic data.

The `ProductPage` uses `StatefulWidget` because its content changes during execution.

Initially, the page is loading. After the API request, the page can change to an error screen or a product list.

The application uses `setState()` to update variables such as `products`, `errorMessage`, and `isLoading`.

This helped me understand that when application state changes, Flutter needs to rebuild the relevant UI so that the user can see the updated information.

The ProductPage therefore connects the API result with the visual state of the application.

7. LEARNING DYNAMIC LIST DISPLAY

I also learned how API data can be displayed dynamically using `ListView.builder`.

Instead of creating a separate widget manually for every product, `ListView.builder` generates the widgets based on the products list.

Each product is displayed using a `Card` and `ListTile`.

The product ID is shown using a `CircleAvatar`, the product name is displayed as the title, and the price is displayed as the subtitle.

This approach is useful because the number of products returned by an API can change. The same code can therefore display a small or large number of products without manually creating each widget.

8. OVERALL LEARNING OUTCOME

Overall, this assignment helped me understand the basic architecture of an API-driven Flutter screen.

The main concepts I learned are:

- Creating and using data models.
- Understanding nullable data.
- Using `Future` for asynchronous operations.
- Using `async` and `await`.
- Simulating API/network delays.
- Managing loading states.
- Handling API errors using `try-catch`.
- Implementing a retry mechanism.
- Handling null and empty responses.
- Using `StatefulWidget` for changing application state.
- Updating the UI using `setState()`.
- Displaying dynamic data using `ListView.builder`.
- Connecting API data with Flutter widgets.

The most important lesson I learned is that an application should be designed to handle different possible outcomes of an API request, not just the successful response.

This assignment gives me a foundation for working with real APIs in future Flutter projects. The same concepts can later be applied to REST APIs, JSON data, authentication, databases, pagination, and more advanced state-management techniques.